

|                            |                     |                 |
|----------------------------|---------------------|-----------------|
| <b>Giáo viên hướng dẫn</b> | <b>Sinh viên</b>    | <b>MSSV</b>     |
| <b>Nguyễn Thành Nhân</b>   | <b>Vũ Đại Dương</b> | <b>23520359</b> |

## CE118-Lab03

### Thiết kế mạch tổ hợp phục vụ tính toán

#### 1. Lý thuyết

**ALU - Arithmetic and Logic Unit** là một mạch tổ hợp để thực hiện các tác vụ về toán học (cộng, trừ, nhân, chia,...) và logic (and, or, not, xor,...).

Một ALU đơn giản sẽ bao gồm 2 phần là khối AU (Arithmetic Unit) chịu trách nhiệm thực hiện các tác vụ về toán học và khối LU (Logic Unit) chịu trách nhiệm thực hiện các tác vụ về logic.

ALU thường sẽ có 2 toán hạng và phép toán được ALU thực hiện sẽ được điều khiển thông qua tín hiệu Opcode.

#### 2. Thực hành

Sinh viên thực hiện thiết kế và mô phỏng một ALU có 2 toán hạng (16 bit) và các phép toán **cộng, cộng 1, trừ, trừ 1, and, or, nand, xor** theo đúng thứ tự tương ứng với tín hiệu điều khiển (Opcode) từ 0→7.

2.1. Ý tưởng thiết kế:

| Opcode | Phép toán |
|--------|-----------|
| 000    | Cộng      |
| 001    | Cộng 1    |
| 010    | Trừ       |
| 011    | Trừ 1     |
| 100    | And       |
| 101    | Or        |
| 110    | Nand      |
| 111    | Xor       |

- Có thể thấy, các phép toán có thể chia ra làm 2 phần:
  - o AU (cộng, cộng 1, trừ, trừ - 1) khi Opcode[2] = 0
  - o LU (And, Or, Nand, Xor) khi Opcode[2] = 1

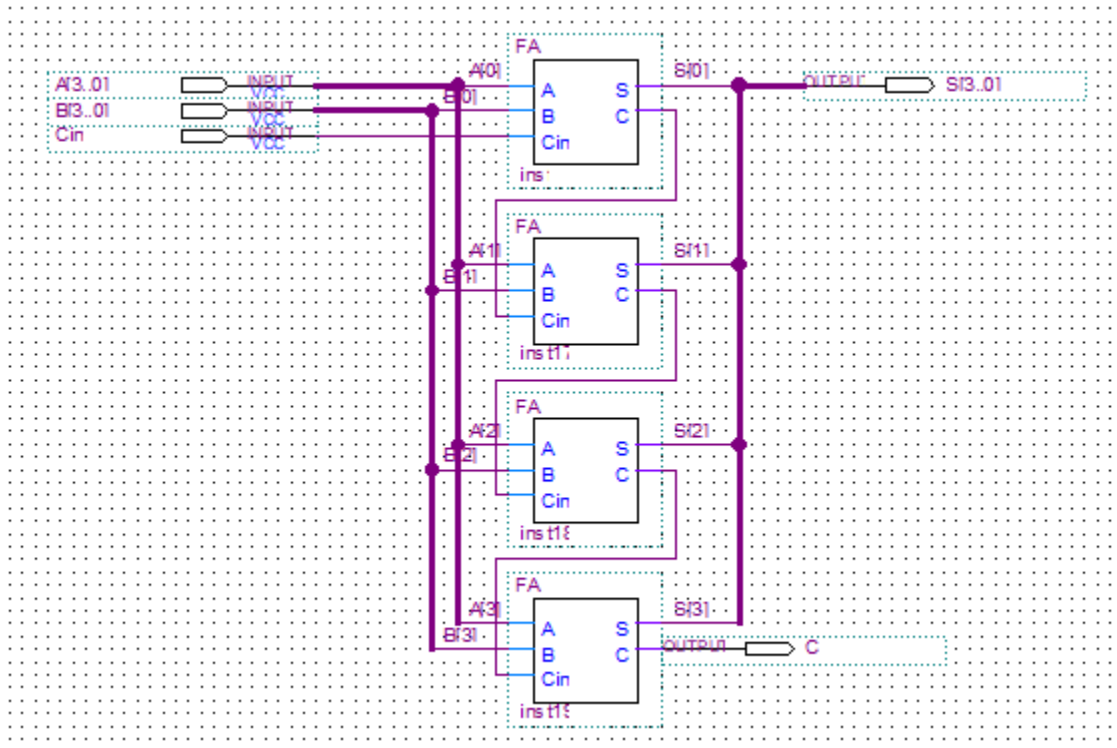
⇒ Có thể dùng MUX21 để chọn 1 trong 2 khối này với select là Opcode[2]
- Tại AU: dùng MUX41 với Select = Opcode[1] và Opcode[0] để chọn phép toán
  - o Cộng:  $A + B$
  - o Cộng 1:  $A + B + 0001_{16}$
  - o Trừ:  $A + B' + 0001_{16}$
  - o Trừ 1:  $A + FFFF_{16}$

- Tại LU: dùng MUX41 với Select = Opcode[1] và Opcode[0] để chọn logic

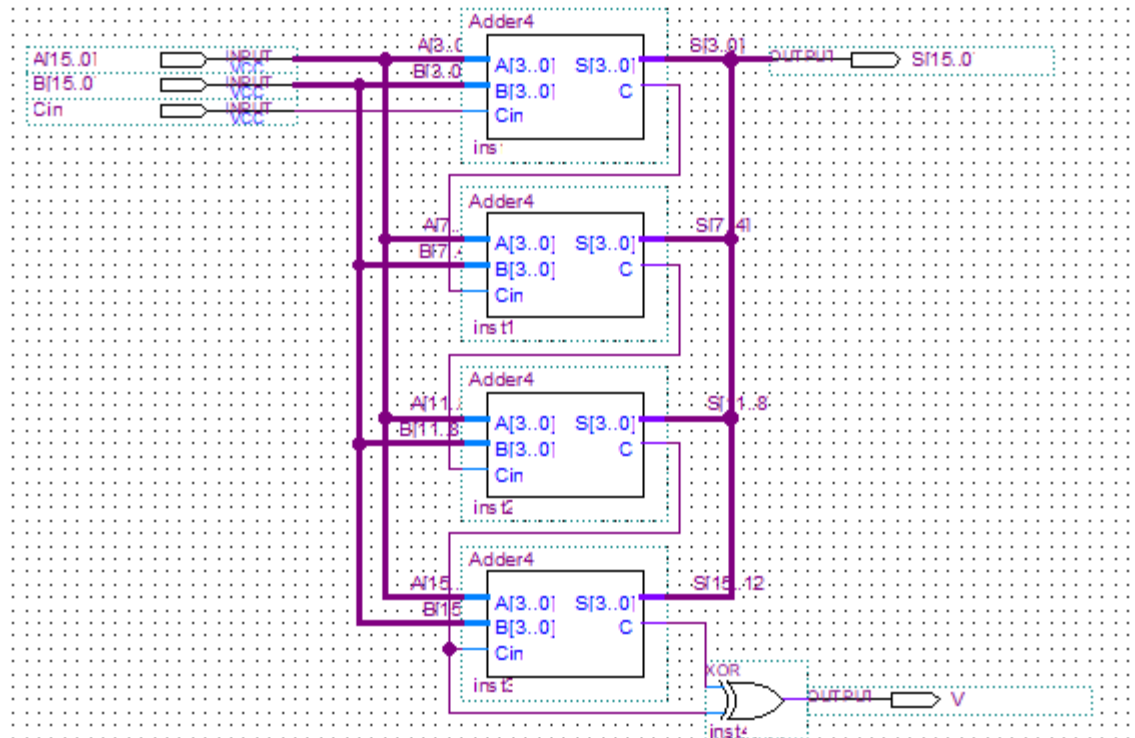
## 2.2. Thiết kế mạch

### 2.2.1. Thiết kế khối AU

- Vì các phép toán được chuyển về phép cộng nên chỉ cần thiết kế bộ Adder 16 bit, sau đó phát triển thành các phép toán
- Thiết kế khối Adder 4 bit:

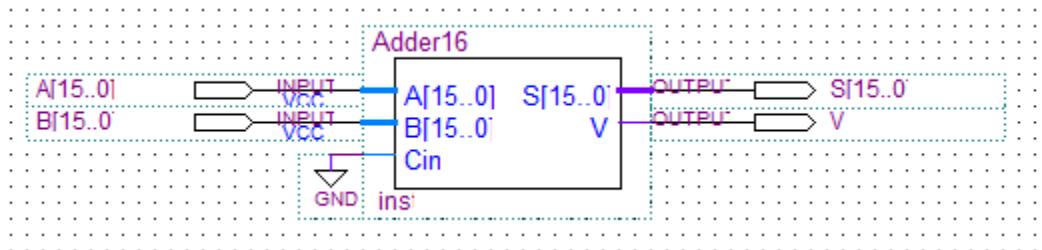


- Thiết kế khối Adder 16 bit từ khối Adder 4 bit:

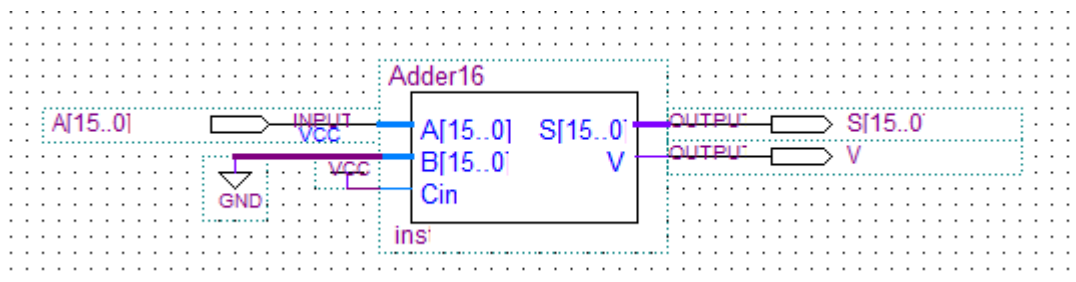


- Thiết kế các phép toán từ Adder 16 bit:

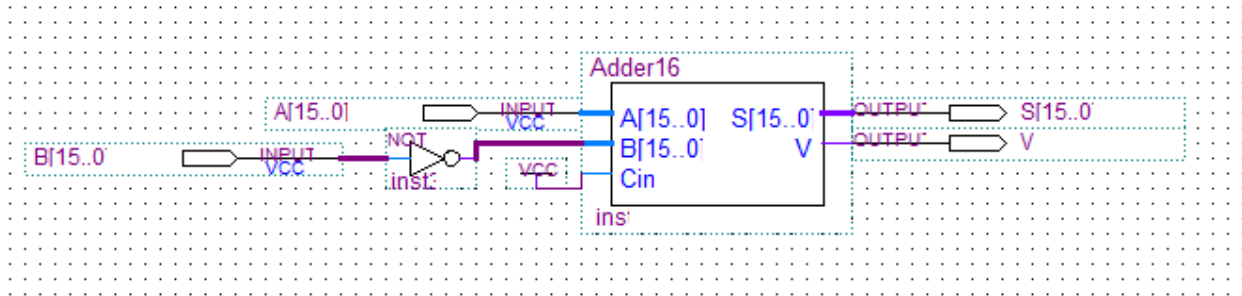
○ Cộng:



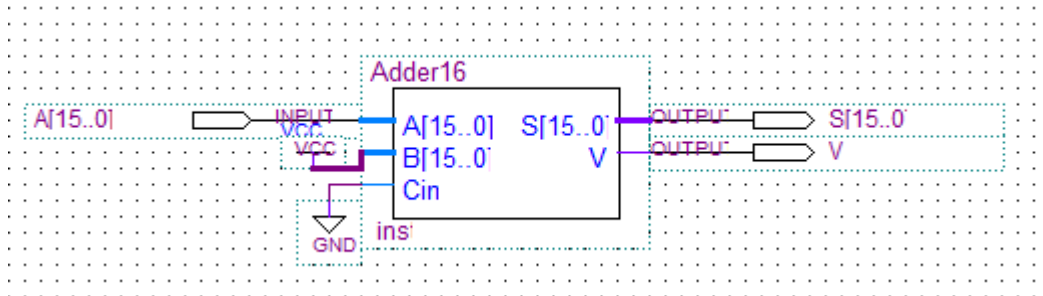
○ Cộng 1:



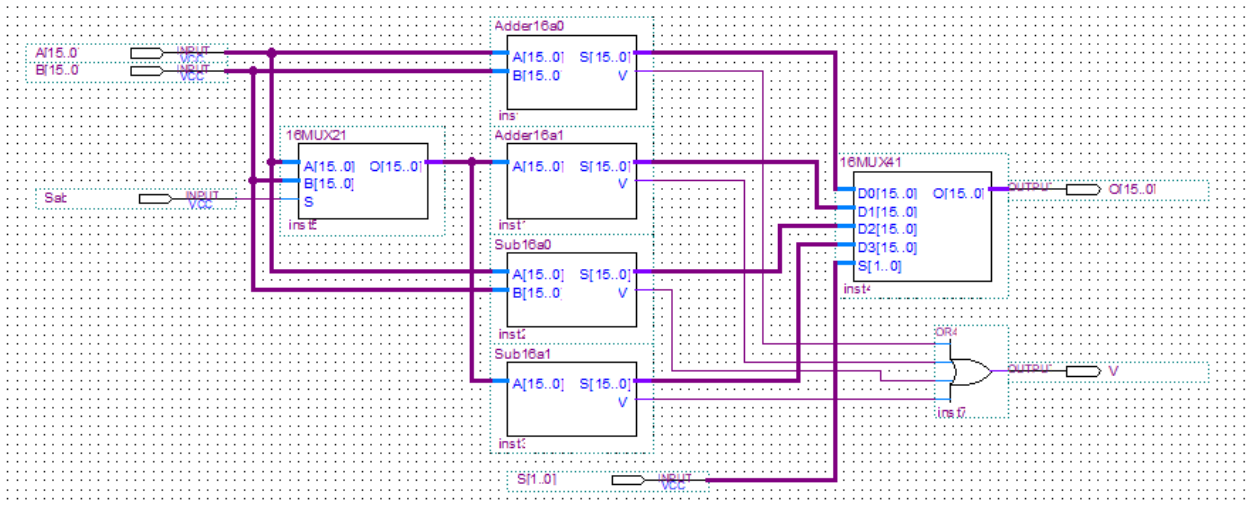
○ Trừ:



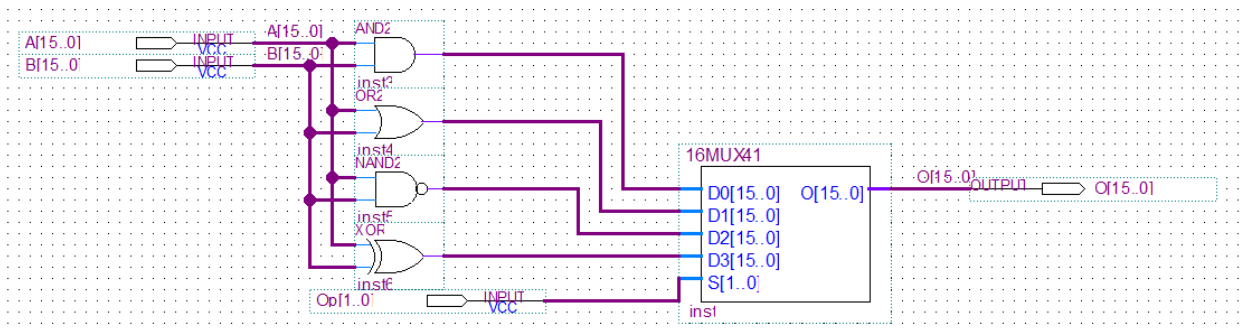
○ Trừ 1:



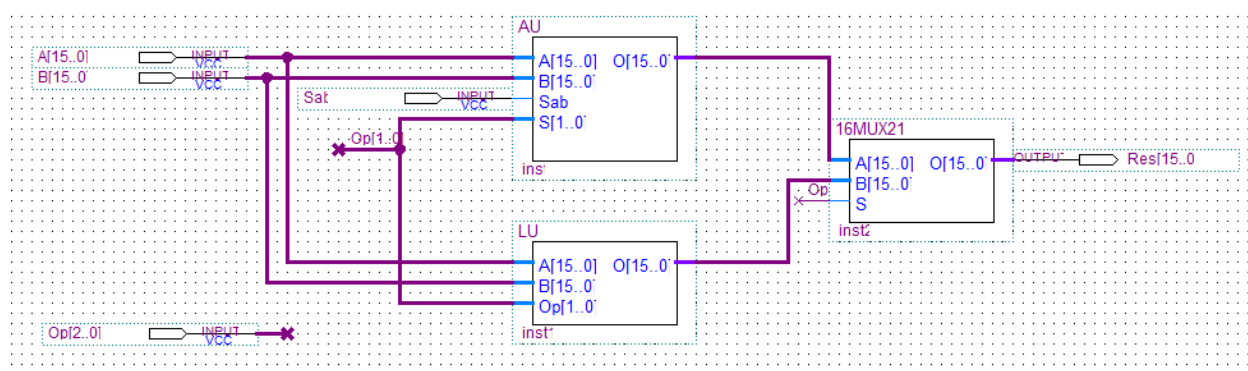
- Thiết kế khối AU từ các khối phép toán:



## 2.2.2. Thiết kế khối LU



### 2.2.3. Thiết kế khối ALU



## 3. Bài tập

Sinh viên thực hiện thiết kế và mô phỏng một bộ nhân 4 bit đơn giản với hệ số nhân là số cuối cùng trong MSSV. Riêng đối với những bạn có số cuối MSSV là 0 (hoặc 1) thì thực hiện bộ nhân 8 (hoặc 9) tương ứng.

Input 4 bit, Output có thể lên tới 8 bit (bởi vì với 2 số 4 bit nhân nhau thì kết quả tối đa cần phải dùng 8 bit để hiển thị)

Ví dụ: Input là số 15 (1111) nhân với 9 (1001) thì kết quả là 135 (cần 8 bit để biểu diễn)

Gợi ý: phép nhân tức là phép dịch bit và phép cộng. Cụ thể, đối với phép nhân 8 thì ta có thể dịch trái 3 bit. Với phép nhân 9 thì ta có thể thực hiện dịch trái 3 bit, sau đó cộng với chính nó (Lưu ý: SV phải thực hiện yêu cầu theo hướng thiết kế bộ dịch và bộ cộng)

### 3.1. Ý tưởng thiết kế:

- Mã số sinh viên: 23520359 → Hệ số nhân là 9
- Theo yêu cầu đề bài, cần:
  - o 1 khối Shift left 3 bit
  - o 1 khối Adder 8 bit (cần tối đa 8 bit)
- Ngoài ra, cần 1 khối Done Check để kiểm tra khối Shift left đã dịch trái 3 bit để khối Adder được hoạt động
- Về bộ Shift left, cần có 2 chức năng là: nạp Input và Shiftleft → cần bộ mux21 để chọn chức năng
- Về bộ Done Check có chức năng so sánh giá trị shift left với giá trị 8 bit=0-I<sub>3</sub>I<sub>2</sub>I<sub>1</sub>I<sub>0</sub>-000
- Về bộ Adder 8 bit có chức năng cộng giá trị đã shift left với giá trị 8 bit có 4 bit thấp là 4 bit Input, và cần 1 tín hiệu để bắt đầu cộng từ khối Done Check.

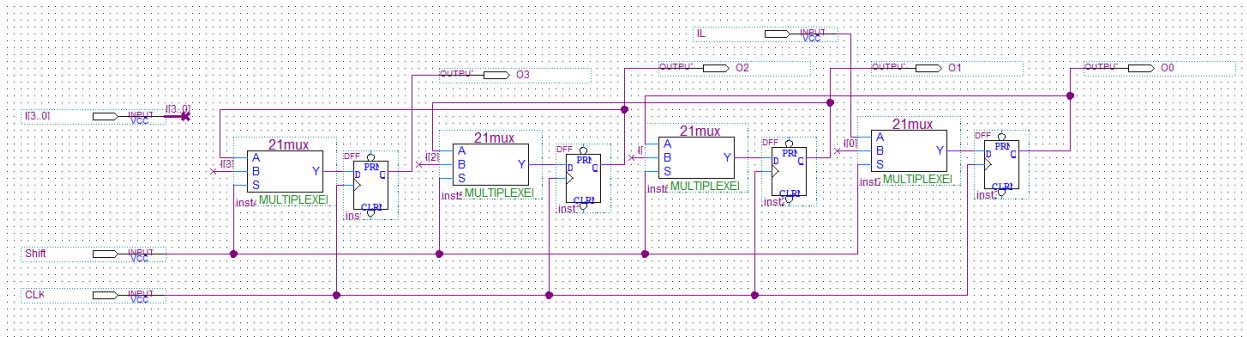
### 3.2. Thiết kế mạch

#### 3.2.1. Thiết kế khối Shift left 8 bit:

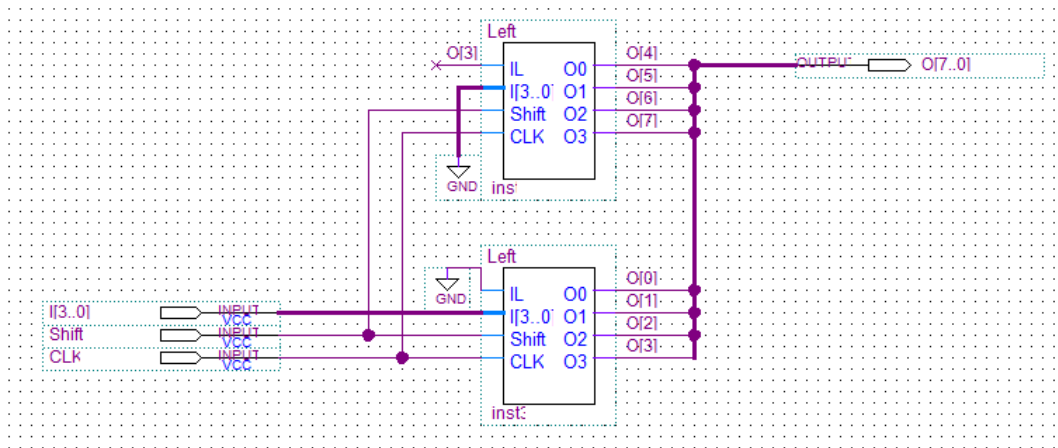
- Thiết kế khối Shift left 4 bit với chức năng nạp Input và Shift left:

| Select | Chức năng |
|--------|-----------|
| 0      | Nạp       |

|   |            |
|---|------------|
| 1 | Shift left |
|---|------------|

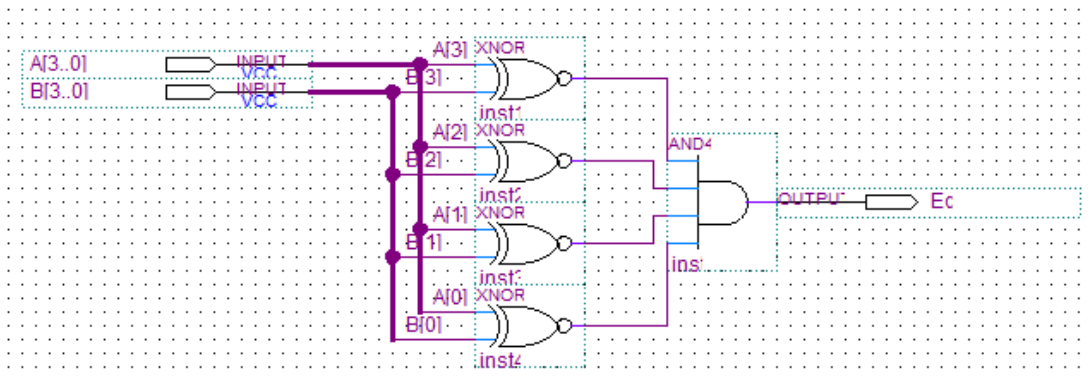


- Thiết kế khối Shift left 8 bit từ khối Shift left 4 bit:

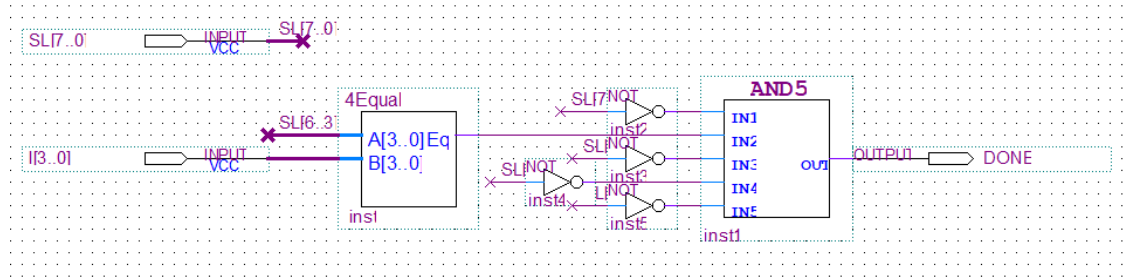


### 3.2.2. Thiết kế khối Done Check

- Thiết kế khối so sánh bằng 4 bit giữa Input và Shift left [6..3]

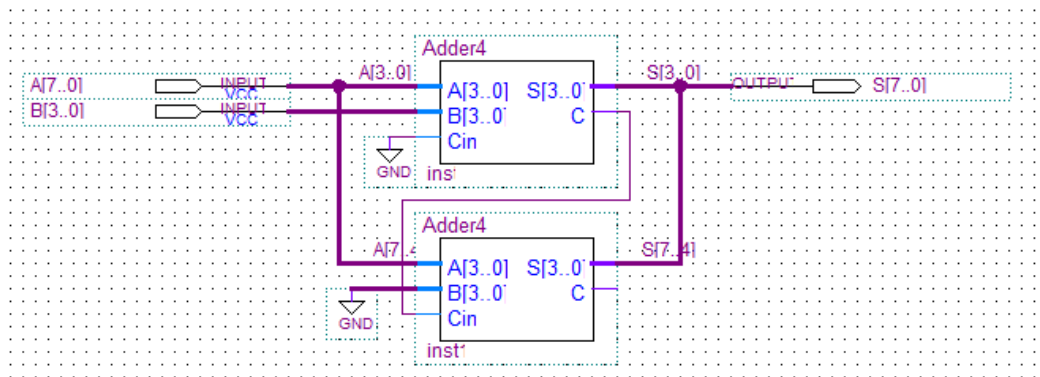


- Thiết kế khối Done Check:

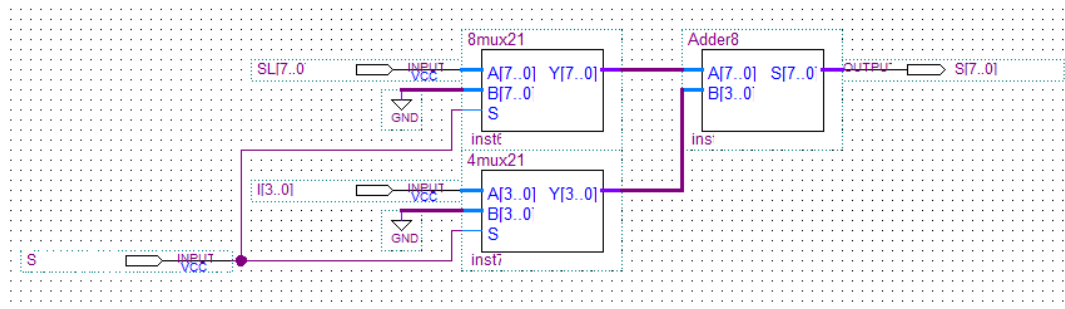


### 3.2.3. Thiết kế khối Adder 8 bit

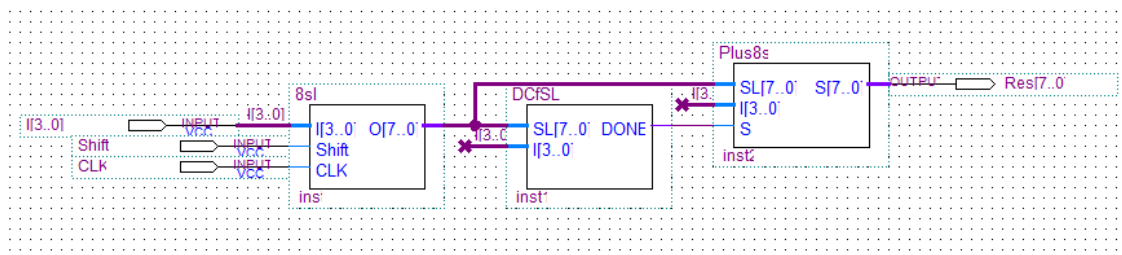
- Thiết kế khối Adder 8 bit với 4 bit từ khối Adder 4 bit:



- Thiết kế khối Adder 8 bit cùng tín hiệu Done Check

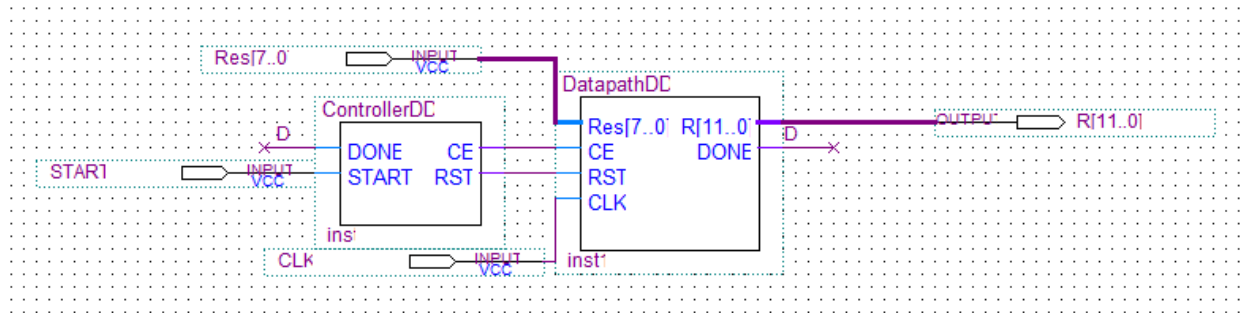


### 3.2.4. Thiết kế khối Multi 9

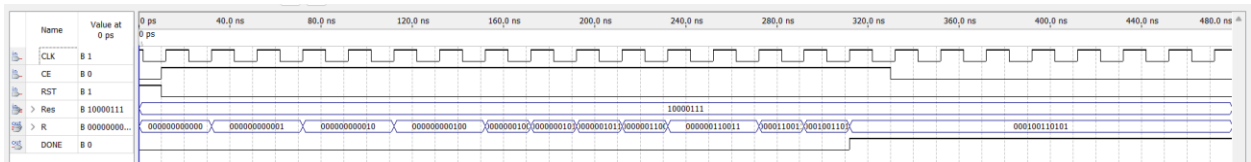
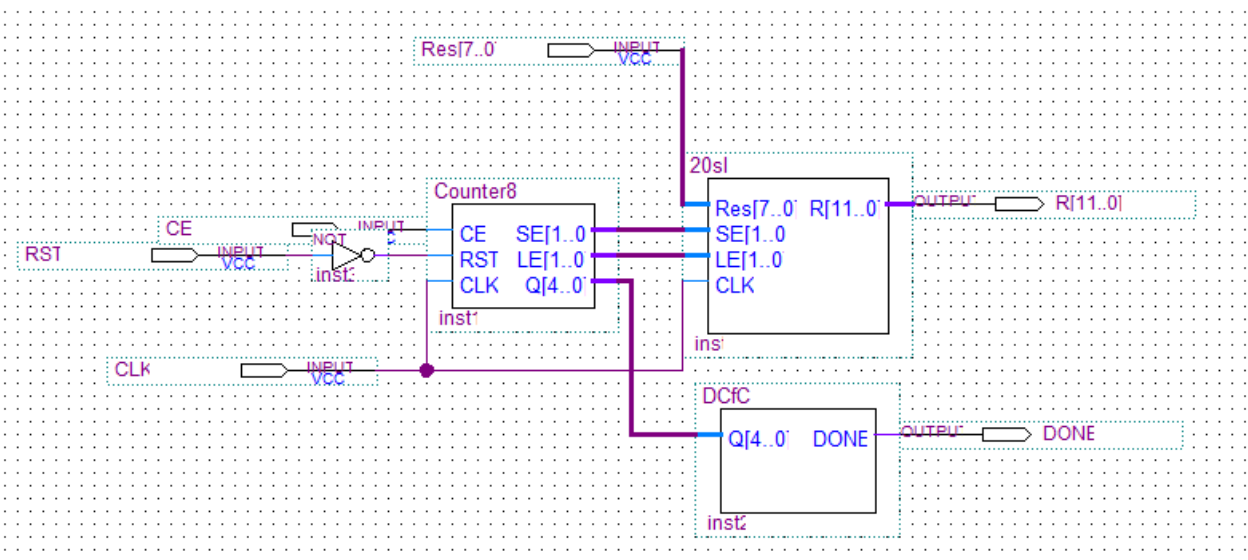


### 3.2.5. Thiết kế khối DD (Double Dabble)

- Để chuyển 8 bit kết quả thành 12 bit để nạp KIT DE2



- DatapathDD:



- o Để in giá trị ra led DE2, mỗi led cần nhận 4 bit giá trị để in ra
- o Input đầu vào: 135 ở 8 bit
- o Output: 000100110101 ở 12 bit với
  - 0001 = 1
  - 0011 = 3
  - 0101 = 5
- ControllerDD:



